

JavaScript

El cerebro de la página

De qué se trata esta guía

Tu página ya tiene esqueleto (HTML) y ropa (CSS), pero todavía no hace nada: es una foto. JavaScript es lo que la vuelve una aplicación. Con él, los botones funcionan, los formularios responden, las cosas aparecen y desaparecen, y la página puede pensar. Esta es la guía donde las tres tecnologías por fin se juntan.

Antes de empezar necesitas:

- Tu carpeta mi-pagina con index.html y estilos.css de las guías 1 y 2.
- Google Chrome, porque vamos a usar mucho su consola.

La parte que más se disfruta

Esta guía tiene una sección entera dedicada a unir las tres tecnologías: ocho proyectos donde HTML pone la estructura, CSS pone el estilo y JavaScript hace que todo funcione. Ahí es donde de verdad se entiende para qué sirve cada uno.

Contenido

- Parte 1 — ¿Qué es JavaScript y dónde se escribe?
- Parte 2 — La consola: tu mejor amiga
- Parte 3 — Variables: guardar información
- Parte 4 — Operaciones y textos
- Parte 5 — Decisiones con if
- Parte 6 — Repetir con for
- Parte 7 — Funciones
- Parte 8 — Listas (arrays)
- Parte 9 — Hablarle al HTML: el DOM
- Parte 10 — Eventos: reaccionar al usuario
- Parte 11 — Leer lo que el usuario escribe
- Parte 12 — Crear elementos desde cero
- Parte 13 — Ocho ejercicios individuales
- Parte 14 — Uniendo las tres tecnologías: ocho proyectos
- Parte 15 — Proyecto final: mi perfil con vida
- Parte 16 — Errores comunes
- Chuleta · Glosario · Lista de verificación

Cómo se usa esta guía

Las partes 1 a 8 son JavaScript puro y se practican en la consola, sin tocar la página. De la parte 9 en adelante empezamos a modificar el HTML de verdad. No te saltes las primeras: son la base de todo lo demás.

Parte 1 — ¿Qué es JavaScript y dónde se escribe?

JavaScript es un lenguaje de programación de verdad: sabe hacer cuentas, tomar decisiones, repetir tareas y recordar cosas. HTML y CSS no programan nada; JavaScript sí.

Lenguaje	Su trabajo	Ejemplo
HTML	Decir qué hay	Aquí hay un botón
CSS	Decir cómo se ve	El botón es azul y redondo
JavaScript	Decir qué pasa	Al hacer clic, aparece un mensaje

1.1 Crear y conectar el archivo

1. En tu carpeta, crea un archivo nuevo llamado script.js
2. Abre index.html y busca la etiqueta de cierre `</body>`.
3. Justo antes de esa etiqueta, escribe:

```
<script src="script.js"></script>
```

4. Guarda los dos archivos.

Va al final del body a propósito, no en el head. La razón es sencilla: el navegador lee de arriba hacia abajo, y si JavaScript se ejecuta antes de que el HTML exista, no va a encontrar los elementos que quiere modificar.

1.2 Comprobar que quedó conectado

Escribe esto en script.js, guarda y abre la página:

```
alert("¡Hola! Ya estoy conectado.");
```

Si aparece una ventanita con el mensaje, funciona. Ahora borra esa línea: alert es útil para probar, pero molesta mucho.

Parte 2 — La consola: tu mejor amiga

La consola es una ventana donde JavaScript te habla. Ahí aparecen tus mensajes y también todos tus errores explicados. Es la herramienta que más vas a usar.

2.1 Abrirla

1. Con tu página abierta en Chrome, presiona la tecla F12.
2. Si no funciona, haz clic derecho y escoge Inspeccionar.
3. En la ventana que se abre, haz clic en la pestaña Console.
4. Puedes escribir directamente ahí y presionar Enter para probar cosas.

2.2 Escribir mensajes desde tu código

```
console.log("Hola desde JavaScript");  
console.log(5 + 3);  
console.log("El resultado es:", 5 + 3);
```

console.log no lo ve el visitante de la página: es solo para ti, para saber qué está pasando por dentro de tu programa.

2.3 Probar cosas ahí mismo

La consola es también una calculadora y un laboratorio. Escribe esto directamente en ella y presiona Enter después de cada línea:

```
2 + 2  
10 * 5  
"Hola" + " " + "mundo"  
"pizza".length  
"grande".toUpperCase()
```

El hábito que separa a quien aprende de quien se frustra

Cuando algo no funcione, no adivines: pon console.log en varios puntos de tu código para ver qué valor tiene cada cosa en cada momento. Los programadores profesionales llenan su código de console.log y después los borran. No es hacer trampa: es el método.

Ejercicio 1 — La consola

Abre la consola y usa JavaScript como calculadora: averigua cuántos minutos tiene un año, cuántos días faltan para tu cumpleaños y cuánto es tu edad multiplicada por 365. Después escribe tres console.log en tu script.js con tu nombre, tu edad y tu comida favorita.

Parte 3 — Variables: guardar información

Una variable es una caja con nombre donde guardas un dato para usarlo después. Se crean así:

```
let nombre = "Camila";
let edad = 12;
const colegio = "San José";

console.log(nombre);
console.log(edad + 1);
```

Palabra	Cuándo se usa
let	Cuando el valor va a cambiar después. Por ejemplo un puntaje.
const	Cuando el valor nunca cambia. Por ejemplo el nombre del colegio.

Usa const por defecto y let solo cuando de verdad necesites cambiar el valor. Así el programa te avisa si cambias algo por accidente.

3.1 Los tipos de dato

```
let texto = "esto es un string, va entre comillas";
let numero = 42;
let decimal = 3.14;
let verdadero = true;
let falso = false;
let nada = null;
```

La diferencia más importante: 5 es un número y "5" es un texto. El primero se puede sumar; el segundo se pega con otro texto. Es la causa de muchísimos errores.

```
5 + 5      da 10
"5" + "5"  da "55"
```

3.2 Reglas para los nombres

- Sin espacios: miNombre está bien, mi nombre no.
- Sin tildes ni ñ, aunque técnicamente el navegador las acepte.
- No pueden empezar con número: nombre1 sí, 1nombre no.
- Las mayúsculas importan: edad y Edad son dos variables distintas.
- Ponles nombres que se entiendan: puntajeTotal es mejor que x.

Ejercicio 2 — Variables

Crea variables para tu nombre, tu edad, tu materia favorita y si te gusta o no el brócoli (true o false). Después escribe un `console.log` que arme una frase completa juntando varias de ellas.

Parte 4 — Operaciones y textos

4.1 Matemáticas

```
let a = 10;
let b = 3;

console.log(a + b);    13    suma
console.log(a - b);    7     resta
console.log(a * b);    30    multiplicación
console.log(a / b);    3.33  división
console.log(a % b);    1     resto de la división
```

El porcentaje no es porcentaje: es el resto de la división. Sirve muchísimo para saber si un número es par, porque `numero % 2` da 0 cuando lo es.

4.2 Cambiar el valor de una variable

```
let puntos = 0;
puntos = puntos + 10;    ahora vale 10
puntos += 5;             forma corta, ahora vale 15
puntos++;               suma 1, ahora vale 16
puntos--;               resta 1, vuelve a 15
```

4.3 Juntar textos

Hay dos formas. La segunda es mucho más cómoda:

```
let nombre = "Camila";
let edad = 12;

Forma antigua, con el signo más:
console.log("Hola " + nombre + ", tienes " + edad);

Forma moderna, con acentos graves y el símbolo del dólar:
console.log(`Hola ${nombre}, tienes ${edad} años`);
```

Los acentos graves están en la tecla que queda a la izquierda del 1, o al lado de la P según el teclado. Es la forma que vas a usar de aquí en adelante porque es mucho más fácil de leer.

4.4 Cosas útiles que se le pueden hacer a un texto

```
let frase = "Me gusta programar";

frase.length           19, cuántas letras tiene
frase.toUpperCase()     todo en mayúsculas
frase.toLowerCase()     todo en minúsculas
frase.includes("gusta") true, si contiene esa palabra
frase.replace("gusta", "encanta")  cambia una palabra
```

Ejercicio 3 — Calculadora personal

Crea variables con tu edad y el año actual. Calcula en qué año naciste, cuántos meses has vivido y cuántos días. Muestra cada resultado con una frase completa usando los acentos graves.

Parte 5 — Decisiones con if

Aquí es donde el programa empieza a pensar. La estructura se lee casi como español:

```
let edad = 12;

if (edad >= 18) {
  console.log("Eres mayor de edad");
} else {
  console.log("Eres menor de edad");
}
```

Lo que va entre paréntesis es una pregunta que solo puede responderse con sí o no. Si es sí, se ejecuta lo del primer bloque; si es no, lo del else.

5.1 Los comparadores

Símbolo	Significa	Ejemplo verdadero
===	es exactamente igual a	5 === 5
!==	es distinto de	5 !== 3
>	es mayor que	10 > 5
<	es menor que	3 < 8
>=	es mayor o igual que	5 >= 5
<=	es menor o igual que	4 <= 9

Cuidado con el igual

Un solo igual (=) guarda un valor en una variable. Tres iguales (===) preguntan si dos cosas son iguales. Escribir uno cuando querías el otro es el error más común de todo principiante en JavaScript.

5.2 Varias condiciones seguidas

```
let nota = 4.2;

if (nota >= 4.5) {
  console.log("Excelente");
} else if (nota >= 3.5) {
  console.log("Bien");
} else if (nota >= 3.0) {
  console.log("Aceptable");
} else {
  console.log("Hay que estudiar más");
}
```

JavaScript revisa de arriba hacia abajo y se detiene en la primera que sea verdadera. Por eso el orden importa.

5.3 Combinar condiciones

```
&&  significa Y: las dos deben cumplirse
||   significa O: basta con que una se cumpla
!    significa NO: invierte la respuesta

if (edad >= 13 && edad <= 19) {
  console.log("Eres adolescente");
}

if (dia === "sábado" || dia === "domingo") {
  console.log("¡Fin de semana!");
}
```

Ejercicio 4 — El clasificador

Escribe un programa que, dada una variable con una temperatura, diga qué ropa ponerse: menos de 10 es abrigo, entre 10 y 20 es chaqueta, entre 20 y 28 es camiseta, y más de 28 es traje de baño. Pruébalo cambiando el valor de la variable.

Parte 6 — Repetir con for

Cuando hay que hacer algo muchas veces, no se copia y pega: se usa un ciclo.

```
for (let i = 1; i <= 5; i++) {  
  console.log(`Vuelta número ${i}`);  
}
```

Los tres pedazos entre paréntesis, separados por punto y coma:

Pedazo	Qué hace
let i = 1	Dónde empieza el contador.
i <= 5	Hasta cuándo sigue repitiendo.
i++	Cuánto suma en cada vuelta.

La letra i viene de índice y es una costumbre de todos los programadores, aunque puedes ponerle el nombre que quieras.

6.1 Ejemplos útiles

```
La tabla del 7:  
for (let i = 1; i <= 10; i++) {  
  console.log(`7 x ${i} = ${7 * i}`);  
}  
  
Contar hacia atrás:  
for (let i = 10; i >= 1; i--) {  
  console.log(i);  
}  
  
Solo los pares:  
for (let i = 0; i <= 20; i += 2) {  
  console.log(i);  
}
```

Cuidado con el ciclo infinito

Si la condición nunca se vuelve falsa, el ciclo no para nunca y el navegador se congela. Pasa cuando se olvida el i++ o cuando la comparación está al revés. Si te ocurre, cierra la pestaña y revisa esos dos puntos.

Ejercicio 5 — Ciclos

Escribe la tabla de multiplicar de tu número favorito, una cuenta regresiva desde 20 hasta 0, y un ciclo que muestre solo los múltiplos de 5 entre 0 y 100.

Parte 7 — Funciones

Una función es un pedazo de código con nombre, que puedes usar cuantas veces quieras. Es como una receta: la escribes una vez y la sigues muchas.

```
function saludar() {  
  console.log("¡Hola a todos!");  
}  
  
saludar();  
saludar();
```

Definir la función no hace nada. Solo cuando la llamas por su nombre con los paréntesis, se ejecuta.

7.1 Funciones que reciben información

```
function saludar(nombre) {  
  console.log(`Hola ${nombre}, qué bueno verte`);  
}  
  
saludar("Camila");  
saludar("Andrés");
```

Lo que va entre paréntesis se llama parámetro. Es información que la función necesita para trabajar, como los ingredientes de una receta.

7.2 Funciones que devuelven un resultado

```
function sumar(a, b) {  
  return a + b;  
}  
  
let resultado = sumar(5, 3);  
console.log(resultado);      8  
console.log(sumar(10, 20));  30
```

La palabra **return** entrega el resultado y termina la función de inmediato. Todo lo que esté escrito después de un **return** nunca se ejecuta.

7.3 Un ejemplo completo

```
function calcularPromedio(n1, n2, n3) {
  let suma = n1 + n2 + n3;
  return suma / 3;
}

function estaAprobado(nota) {
  if (nota >= 3) {
    return true;
  } else {
    return false;
  }
}

let promedio = calcularPromedio(4.5, 3.8, 4.0);
console.log(`Promedio: ${promedio}`);
console.log(`¿Aprobó? ${estaAprobado(promedio)}`);
```

Ejercicio 6 — Funciones

Escribe cuatro funciones: una que reciba un número y diga si es par, una que reciba dos números y devuelva el mayor, una que reciba una palabra y la devuelva al revés, y una que reciba tu edad y devuelva cuántos días has vivido.

Parte 8 — Listas (arrays)

Un array es una variable que guarda muchos valores en orden, dentro de corchetes:

```
let frutas = ["manzana", "banano", "mango"];
let notas = [4.5, 3.8, 5.0, 2.9];

console.log(frutas[0]);      manzana
console.log(frutas[2]);      mango
console.log(frutas.length);  3
```

Se empieza a contar desde **CERO**, no desde uno. El primer elemento es el número 0. Es rarísimo al principio y luego se vuelve natural.

8.1 Modificar una lista

```
frutas.push("pera");      agrega al final
frutas.pop();             quita el último
frutas[0] = "fresa";      cambia el primero
frutas.includes("mango")  true si está en la lista
frutas.indexOf("mango")   en qué posición está
```

8.2 Recorrer una lista

```
Con for de toda la vida:
for (let i = 0; i < frutas.length; i++) {
  console.log(frutas[i]);
}
```

```
Con forEach, más corto y más claro:
frutas.forEach(function (fruta) {
  console.log(fruta);
});
```

8.3 Un ejemplo con todo junto

```
let notas = [4.5, 3.8, 5.0, 2.9, 4.1];
let suma = 0;

for (let i = 0; i < notas.length; i++) {
  suma += notas[i];
}

let promedio = suma / notas.length;
console.log(`Promedio: ${promedio}`);
```

Ejercicio 7 — Arrays

Crea una lista con los nombres de cinco compañeros. Muéstralos todos en la consola con un ciclo. Después agrega uno nuevo, elimina el último y muestra cuántos quedaron. Por último, crea una lista de números y calcula cuál es el mayor.

Parte 9 — Hablarle al HTML: el DOM

Hasta aquí JavaScript solo hablaba con la consola. Ahora empieza lo bueno: modificar la página de verdad.

El DOM es la manera en que JavaScript ve tu HTML: como una lista de elementos que puede buscar, leer y cambiar.

9.1 Buscar un elemento

```
En el HTML:
<h1 id="titulo">Hola</h1>
<p class="mensaje">Texto uno</p>

En el JavaScript:
let titulo = document.getElementById("titulo");
let mensaje = document.querySelector(".mensaje");
let todos = document.querySelectorAll(".mensaje");
```

Forma de buscar	Qué encuentra
getElementById	El elemento que tenga ese id exacto.
querySelector	El primero que coincida. Se escribe como en CSS: con punto para clases.
querySelectorAll	Todos los que coincidan, en una lista.

querySelector usa exactamente la misma sintaxis que los selectores de CSS que aprendiste en la guía 2. Punto para clases, numeral para ids. Es el mismo conocimiento, reutilizado.

9.2 Cambiar lo que dice

```
let titulo = document.getElementById("titulo");

titulo.textContent = "Nuevo título";
titulo.innerHTML = "Título con <strong>negrilla</strong>";

console.log(titulo.textContent); leer lo que dice
```

textContent pone texto plano; innerHTML permite meter etiquetas. Usa textContent siempre que puedas: es más seguro.

9.3 Cambiar cómo se ve

Cambiando el estilo directamente:

```
titulo.style.color = "red";  
titulo.style.fontSize = "50px";  
titulo.style.display = "none";
```

Mejor: agregando o quitando una clase de CSS

```
titulo.classList.add("destacado");  
titulo.classList.remove("destacado");  
titulo.classList.toggle("destacado");
```

La segunda forma es la profesional: el estilo se queda en el archivo CSS donde debe estar, y JavaScript solo decide cuándo aplicarlo. **toggle** es especialmente útil: pone la clase si no está y la quita si está.

Ojo con una diferencia: en CSS se escribe font-size con guion, pero en JavaScript se escribe fontSize sin guion y con mayúscula. Lo mismo con background-color, que pasa a ser backgroundColor.

Ejercicio 8 — Manipular la página

En tu página de perfil, usa JavaScript para cambiar el texto de tu h1, ponerle otro color a un párrafo y esconder una sección con display none. Todo desde script.js, sin tocar el HTML.

Parte 10 — Eventos: reaccionar al usuario

Un evento es algo que pasa: un clic, una tecla, mover el mouse. JavaScript puede quedarse esperando a que ocurra.

```
En el HTML:
<button id="miBoton">Haz clic</button>
<p id="salida"></p>

En el JavaScript:
let boton = document.getElementById("miBoton");
let salida = document.getElementById("salida");

boton.addEventListener("click", function () {
  salida.textContent = "¡Me hiciste clic!";
});
```

Se lee así: al botón, agrégle un escuchador del evento clic, y cuando ocurra, haz lo que está adentro de la función.

10.1 Los eventos más usados

Evento	Cuándo ocurre
click	Al hacer clic en el elemento.
input	Cada vez que se escribe una letra en una casilla.
change	Cuando se cambia el valor de un select o una casilla.
submit	Al enviar un formulario.
keydown	Al presionar una tecla.
mouseover	Al pasar el mouse por encima.

10.2 Un contador completo

```
HTML:
<p id="marcador">0</p>
<button id="sumar">+1</button>
<button id="reiniciar">Reiniciar</button>

JavaScript:
let cuenta = 0;
let marcador = document.getElementById("marcador");

document.getElementById("sumar")
  .addEventListener("click", function () {
    cuenta++;
    marcador.textContent = cuenta;
  });

document.getElementById("reiniciar")
  .addEventListener("click", function () {
    cuenta = 0;
    marcador.textContent = cuenta;
  });
```

Fíjate en el patrón que se repite en todo JavaScript de páginas web: buscar el elemento, escuchar el evento, cambiar algo. Todo lo demás son variaciones de eso.

Parte 11 — Leer lo que el usuario escribe

Aquí es donde los formularios de la guía 1 por fin sirven para algo.

```
HTML:
<input type="text" id="nombre" placeholder="Tu nombre">
<button id="saludar">Saludar</button>
<h2 id="saludo"></h2>

JavaScript:
document.getElementById("saludar")
  .addEventListener("click", function () {
    let nombre = document.getElementById("nombre").value;
    let saludo = document.getElementById("saludo");
    saludo.textContent = `¡Hola ${nombre}!`;
  });
```

La palabra clave es `.value`: eso es lo que la persona escribió en la casilla. Para el texto de un párrafo se usa `textContent`, pero para una casilla de formulario se usa `value`.

11.1 Los números llegan como texto

Aunque el input sea de tipo number, lo que llega es un texto. Hay que convertirlo:

```
let edad = document.getElementById("edad").value;
console.log(edad + 1);           si escribió 12, da "121"

Convertido a número:
let edad = Number(document.getElementById("edad").value);
console.log(edad + 1);           ahora sí da 13
```

Este error hace perder horas a mucha gente. Si sumas dos cosas y salen pegadas en vez de sumadas, es esto.

11.2 Validar antes de usar

```
let nombre = document.getElementById("nombre").value;

if (nombre === "") {
  alert("Por favor escribe tu nombre");
} else {
  saludo.textContent = `¡Hola ${nombre}!`;
}
```

Ejercicio 9 — Formulario que responde

Haz una página donde la persona escriba su nombre y su edad, y al presionar el botón aparezca un mensaje que diga cuántos años tendrá en el 2050. Valida que no haya dejado las casillas vacías.

Parte 12 — Crear elementos desde cero

JavaScript no solo cambia lo que existe: puede crear HTML nuevo mientras la página está abierta.

```
let lista = document.getElementById("lista");

let nuevo = document.createElement("li");
nuevo.textContent = "Elemento nuevo";
lista.appendChild(nuevo);
```

Son tres pasos: crear el elemento, ponerle contenido y meterlo dentro de otro. Es como armar una pieza y después pegarla.

12.1 Dibujar una lista completa

```
let tareas = ["Estudiar", "Entrenar", "Leer"];
let lista = document.getElementById("lista");

tareas.forEach(function (tarea) {
  let li = document.createElement("li");
  li.textContent = tarea;
  lista.appendChild(li);
});
```

12.2 El patrón de volver a dibujar

Cuando una lista cambia, lo más fácil es borrarla entera y dibujarla de nuevo. Se ve así:

```
let tareas = [];
let lista = document.getElementById("lista");

function dibujar() {
  lista.innerHTML = "";
  tareas.forEach(function (tarea) {
    let li = document.createElement("li");
    li.textContent = tarea;
    lista.appendChild(li);
  });
}

document.getElementById("agregar")
  .addEventListener("click", function () {
    let texto = document.getElementById("nueva").value;
    if (texto !== "") {
      tareas.push(texto);
      document.getElementById("nueva").value = "";
      dibujar();
    }
  });
```

Este patrón (guardar los datos en un array y tener una función que dibuja) es la base de casi todas las aplicaciones web modernas. Si lo entiendes, entendiste algo grande.

La línea que pone value igual a comillas vacías limpia la casilla después de agregar. Es un detalle pequeño que hace que la aplicación se sienta bien hecha.

Parte 13 — Ocho ejercicios individuales

Estos son solo de JavaScript y se resuelven en la consola o con `console.log`. Sin diseño, sin página: solo lógica.

A — El conversor

Una función que reciba grados centígrados y devuelva los fahrenheit. La fórmula es centígrados por 9, dividido 5, más 32.

B — El par o impar

Una función que reciba un número y devuelva el texto par o impar. Pista: usa el operador %.

C — La tabla completa

Un programa que muestre las tablas de multiplicar del 1 al 10, todas. Pista: necesitas un ciclo dentro de otro ciclo.

D — El mayor de la lista

Dado un array de números, encuentra el más grande sin usar funciones especiales, solo con un ciclo y un `if`.

E — El contador de vocales

Una función que reciba una palabra y diga cuántas vocales tiene. Pista: recorre letra por letra con un ciclo y usa `includes`.

F — El adivinador

El programa escoge un número secreto del 1 al 10 con `Math.floor(Math.random() * 10) + 1`. El usuario adivina con `prompt` y el programa dice si es mayor o menor, hasta que acierte.

G — La calculadora de promedio

Un array con las notas de un estudiante. Calcula el promedio, di si aprobó y cuenta cuántas notas fueron mayores a 4.

H — Detective de errores

Este código tiene cinco errores. Encuéntralos:

```
let nombre = Camila;

function saludar(nombre) {
  console.log(`Hola ${nombre}`)

if (edad = 12) {
  console.log("Tiene 12")
}

for (let i = 0; i < 5; i--) {
  console.log(i);
}
```

Pista: un texto sin comillas, una llave sin cerrar, un igual que debería ser triple, una variable que no existe y un ciclo infinito.

Parte 14 — Uniendo las tres tecnologías

Estos son los proyectos importantes. Cada uno se hace en una carpeta nueva con tres archivos: index.html, estilos.css y script.js. Están ordenados de más fácil a más difícil.

El reparto de trabajo, siempre igual

HTML pone los elementos y les da un id. CSS los pinta y define las clases especiales. JavaScript busca esos ids, escucha los eventos y agrega o quita esas clases. Si tienes clara esa división, cualquier proyecto se vuelve armable.

Proyecto 1 — El interruptor de luz

Dificultad: Fácil

- **HTML:** un div grande que sea la bombilla y un botón.
- **CSS:** la bombilla gris por defecto, y una clase llamada encendida que la pone amarilla con una sombra brillante. Agrega transition para que el cambio sea suave.
- **JS:** al hacer clic en el botón, usa `classList.toggle` para poner o quitar la clase encendida.

Este proyecto en tres líneas de JavaScript enseña la idea más importante de todas: JavaScript no pinta, solo decide cuándo aplicar lo que CSS ya definió.

Proyecto 2 — El semáforo

Dificultad: Fácil

- **HTML:** tres divs circulares dentro de una caja, más un botón de Cambiar.
- **CSS:** los círculos apagados en gris oscuro, y clases rojo, amarillo y verde con su color encendido.
- **JS:** una variable que guarde el estado actual y un if que decida cuál encender en cada clic, quitando la clase de los otros dos.
- **Reto:** que cambie solo cada tres segundos usando `setInterval`.

Proyecto 3 — La lista de tareas

Dificultad: Media · *Es el proyecto clásico y el más completo de todos.*

- **HTML:** un input, un botón de agregar y un ul vacío con id.
- **CSS:** las tareas como tarjetas, y una clase hecha que las tache y las ponga grises.
- **JS:** un array de tareas, la función dibujar de la parte 12, y que al hacer clic en una tarea se le ponga la clase hecha.
- **Reto:** agregar un botón de borrar a cada tarea y un contador de cuántas faltan.

Proyecto 4 — La calculadora de propina

Dificultad: Media

- **HTML:** un input para el valor de la cuenta, otro para el número de personas, y un párrafo para el resultado.
- **CSS:** un formulario centrado en una tarjeta, con el resultado grande y de color.
- **JS:** leer los dos valores con `.value`, convertirlos con `Number`, calcular el 10% y dividir entre las personas.
- **Cuidado:** si no conviertes con `Number`, los valores se pegan en vez de sumarse. Usa `toFixed(2)` para mostrar solo dos decimales.

Proyecto 5 — El juego de adivinar el número

Dificultad: Media

- **HTML:** un input de número, un botón de adivinar, un párrafo de pistas y un contador de intentos.
- **CSS:** clases pista-alta, pista-baja y acierto con colores distintos para el mensaje.
- **JS:** generar el número secreto al cargar, comparar en cada clic, contar intentos y cambiar la clase del mensaje según el resultado.
- **Reto:** un botón de jugar de nuevo que reinicie todo sin recargar la página.

Proyecto 6 — La galería con filtros

Dificultad: Media alta

- **HTML:** ocho tarjetas con imágenes, cada una con una clase de categoría (animales, deportes, comida), y tres botones de filtro.
- **CSS:** la galería con Flexbox y flex-wrap, más una clase oculta con display none.
- **JS:** al hacer clic en un filtro, recorrer todas las tarjetas con querySelectorAll y ponerle o quitarle la clase oculta a cada una según su categoría.

Proyecto 7 — El quiz de preguntas

Dificultad: Difícil

- **HTML:** un contenedor para la pregunta, otro para los botones de respuesta y un marcador de puntos.
- **CSS:** clases correcta (verde) e incorrecta (roja) para los botones después de responder.
- **JS:** un array de preguntas donde cada una es un objeto con el texto, las opciones y cuál es la correcta. Crear los botones con createElement y avanzar a la siguiente al responder.

```
let preguntas = [  
  {  
    texto: "¿Cuál es la capital de Colombia?",  
    opciones: ["Cali", "Bogotá", "Medellín"],  
    correcta: 1  
  }  
];
```

Proyecto 8 — El buscador con filtro en vivo

Dificultad: Difícil

- **HTML:** un input de búsqueda y una lista larga de elementos (países, películas, compañeros).
- **CSS:** la lista con estilo de tarjetas y la clase oculta.

- **JS:** escuchar el evento input (no el click), y en cada letra que se escriba, mostrar solo los elementos cuyo texto incluya lo escrito. Usa toLowerCase en los dos lados para que no importen las mayúsculas.

Este es el que más impresiona: al escribir, la lista se filtra al instante. Y es de los más cortos de todos.

Parte 15 — Proyecto final: mi perfil con vida

Volvemos a tu página de perfil, la misma de las guías 1 y 2. Ahora le vamos a dar cerebro. Debe cumplir con al menos seis de estas ocho funciones:

Función	Qué debe hacer
Modo oscuro	Un botón que cambie toda la página a colores oscuros y de vuelta, con <code>classList.toggle</code> en el body.
Saludo por hora	Que diga Buenos días, Buenas tardes o Buenas noches según la hora real. Usa <code>new Date().getHours()</code> .
Menú que funciona	Que al hacer clic en cada enlace del nav se muestre solo esa sección y se oculten las demás.
Contador de visitas	Un número que aumente cada vez que se hace clic en tu foto.
Formulario que responde	Que al enviar aparezca un mensaje de gracias con el nombre que escribieron.
Lista que crece	Una sección donde puedas agregar tus gustos escribiéndolos en una casilla.
Frase del día	Un array de frases y un botón que muestre una al azar.
Mostrar y ocultar	Un botón de Ver más que despliegue un párrafo escondido.

Lo que acabas de lograr

Cuando termines, tendrás una página web completa, hecha por ti desde la primera línea: estructurada con HTML, diseñada con CSS y programada con JavaScript. Eso es exactamente lo mismo que hace un desarrollador web profesional. La escala es distinta, las herramientas son las mismas.

Parte 16 — Errores comunes

El error dice	Qué significa y cómo se arregla
Cannot read properties of null	El elemento no se encontró. Revisa que el id esté escrito igual en el HTML y en el JS, y que el script esté al final del body.
X is not defined	Estás usando una variable que no existe, o la escribiste con distinta mayúscula.
X is not a function	Escribiste mal el nombre. Recuerda: <code>addEventListener</code> , <code>textContent</code> , <code>querySelector</code> .
Unexpected token / Unexpected end	Falta cerrar una llave, un paréntesis o unas comillas.
Los números se pegan en vez de sumarse	Falta convertir con <code>Number</code> lo que sacaste del input.
El botón no hace nada	Revisa la consola. Casi siempre el id no coincide o hay un error más arriba que detuvo todo.
La página se congeló	Un ciclo infinito. Cierra la pestaña y revisa la condición del <code>for</code> .
Funciona una vez y luego no	Estás reasignando algo que era <code>const</code> , o la variable se reinicia porque está dentro de la función.

Cómo se lee un error de JavaScript

En la consola, el mensaje rojo trae tres cosas: qué pasó, en qué archivo y en qué línea. Haz clic en el número de línea y Chrome te lleva exactamente ahí. Casi siempre el problema está en esa línea o en la anterior. Leer errores no es un castigo: es la habilidad principal del oficio.

Chuleta de JavaScript

Instrucción	Para qué
<code>let / const</code>	Crear una variable que cambia / que no cambia
<code>console.log()</code>	Escribir un mensaje en la consola
<code>if / else if / else</code>	Tomar decisiones
<code>=== / !== / > / <</code>	Comparar valores
<code>&& / / !</code>	Y / O / NO
<code>for (let i = 0; ...)</code>	Repetir un número de veces
<code>function nombre() { }</code>	Definir una función
<code>return</code>	Devolver un resultado
<code>[] .push() .length</code>	Listas: crear, agregar, contar
<code>forEach()</code>	Recorrer una lista
<code>getElementById()</code>	Buscar un elemento por su id
<code>querySelector()</code>	Buscar con sintaxis de CSS
<code>textContent / .value</code>	Texto de un elemento / valor de una casilla
<code>classList.add / remove / toggle</code>	Agregar, quitar o alternar una clase
<code>addEventListener()</code>	Escuchar un evento
<code>createElement / appendChild</code>	Crear un elemento y meterlo en la página
<code>Number()</code>	Convertir un texto en número

Glosario

Palabra	Qué significa
Variable	Una caja con nombre donde se guarda un dato.
String	Un texto. Va entre comillas.
Booleano	Un valor que solo puede ser true o false.
Array	Una lista ordenada de valores. Empieza a contar en cero.
Función	Un bloque de código con nombre que se puede reutilizar.
Parámetro	La información que recibe una función.
Ciclo	Repetir instrucciones varias veces.
DOM	La forma en que JavaScript ve el HTML de la página.
Evento	Algo que ocurre: un clic, una tecla, un cambio.
Consola	La ventana donde JavaScript muestra mensajes y errores.

Lista de verificación

- ☐ Creé script.js y lo conecté al final del body.
- ☐ Sé abrir la consola y leer los errores.
- ☐ Entiendo la diferencia entre let y const.
- ☐ Entiendo por qué "5" + 5 no da 10.
- ☐ Escribí condiciones con if y comparé con triple igual.
- ☐ Hice ciclos con for sin quedar en bucle infinito.
- ☐ Escribí funciones con parámetros y return.

- ☐ Recorrí un array y encontré el mayor.
- ☐ Cambié el texto de un elemento con `textContent`.
- ☐ Usé `classList.toggle` para prender y apagar algo.
- ☐ Leí lo que un usuario escribió con `.value`.
- ☐ Creé elementos nuevos con `createElement`.
- ☐ Terminé al menos cuatro proyectos de la parte 14.
- ☐ Le di vida a mi página de perfil.

Esqueleto, ropa y cerebro. Ya sabes construir la web.